The Adjoint Method: Origins, Derivations, and Practical Use*

January 20, 2026

Contents

1 Motivation and Big Picture **2**

2 Lagrangian View and First-Order Conditions **2**

3 Adjoints for ODE-Constrained Problems (Pontryagin) **3**

4 Adjoints for PDE-Constrained Problems **3**

5 Discrete vs. Continuous Adjoint **4**

6 Adjoint and Algorithmic Differentiation **4**

7 Inequality Constraints and KKT **4**

8 Second-Order Information and Newton-Krylov **5**

9 Algorithmic Pattern **5**

10 Worked Examples **5**

10.1 Linear-Quadratic Regulator (discrete time) 5

10.2 Elliptic PDE Control (continuous time) 6

11 Repository-Specific Mapping (External Mesh Ex3) **6**

11.1 Model and Spaces 6

11.2 Discrete Operators 6

11.3 State, Adjoint, and Objective 6

11.4 Reduced Gradient and Hessian 7

11.5 Algorithmic Details and Verification 7

12 Boundary/Terminal Conditions and Transversality **7**

13 Practical Tips **7**

*Prepared as an extended note on adjoints for optimal control and PDE-constrained optimization.

14 Summary **8**

Abstract

The adjoint method provides gradients and sensitivities for systems constrained by differential equations with a computational cost that is essentially independent of the dimension of the control or parameter space. This report derives the adjoint equations from first principles in calculus of variations and Lagrangian/KKT theory, presents both ODE- and PDE-based formulations (Pontryagin-type optimality systems), and discusses discrete vs. continuous adjoints, algorithmic differentiation, second-order information via incremental adjoints, and implementation patterns. We include canonical examples (linear-quadratic, elliptic PDE control) and outline how adjoints drive modern gradient-based solvers in large-scale optimization.

1 Motivation and Big Picture

Consider an optimization problem constrained by a differential equation (ODE or PDE): find controls/parameters u that minimize an objective depending on the corresponding state y:

$$\min_u \; J(y,u) \quad \text{subject to} \quad C(y,u) = 0. \tag{1.1}$$

Here $C(y,u) = 0$ models the governing dynamics (e.g., a PDE) and implicitly defines the parameter-to-state map $u \mapsto y(u)$. The reduced objective is $\hat{J}(u) := J(y(u),u)$. To use gradient-based optimization efficiently, we need $\nabla \hat{J}(u)$.

A naive finite-difference gradient costs one forward solve per control component, which is prohibitive when $\dim(u)$ is large. The adjoint method yields $\nabla \hat{J}(u)$ at a cost comparable to *one* additional solve (the adjoint), independent of $\dim(u)$. This is achieved by introducing a Lagrange multiplier (the adjoint variable) and shifting derivatives via integration by parts or transposition of Jacobians.

2 Lagrangian View and First-Order Conditions

Let Y, U be appropriate state and control spaces, and $C : Y \times U \to Z$ encode the constraint. Define the Lagrangian with adjoint $p \in Z$:

$$\mathcal{L}(y,u,p) := J(y,u) + \langle p,\, C(y,u) \rangle \,. \tag{2.1}$$

Formally, the first-order necessary optimality (KKT) conditions are

$$\text{State:} \quad C(y,u) = 0, \tag{2.2a}$$

$$\text{Adjoint:} \quad C_y(y,u)^* p + J_y(y,u) = 0, \tag{2.2b}$$

$$\text{Gradient:} \quad \nabla \hat{J}(u) = J_u(y,u) + C_u(y,u)^* p = 0 \quad \text{(unconstrained)}. \tag{2.2c}$$

If u is constrained (e.g., box or inequality constraints), the gradient condition is replaced by a variational inequality or projection (see Section 7). The operator $\cdot^*$ denotes the adjoint

(transpose in finite dimensions). Equation (2.2) shows how the adjoint p enters the reduced gradient.

3 Adjoints for ODE-Constrained Problems (Pontryagin)

Consider the optimal control problem

$$\min_{u(\cdot)} \; J(y,u) := \int_0^T \ell(y(t),u(t),t)\,\mathrm{d}t + \phi(y(T)) \tag{3.1}$$

subject to the state ODE

$$\dot{y}(t) = f(y(t),u(t),t), \quad y(0) = y_0. \tag{3.2}$$

Define the Hamiltonian $H(y,u,p,t) := \ell(y,u,t) + p^{\mathrm{T}} f(y,u,t)$. The Pontryagin minimum principle states that a necessary condition for optimality is the existence of an adjoint $p(t)$ such that

$$\dot{y} = f(y,u,t), \qquad y(0) = y_0, \tag{3.3a}$$

$$-\dot{p} = H_y(y,u,p,t) = \ell_y + f_y^{\mathrm{T}} p, \qquad p(T) = \phi_y(y(T)), \tag{3.3b}$$

$$0 = H_u(y,u,p,t) = \ell_u + f_u^{\mathrm{T}} p \quad \text{(unconstrained)}. \tag{3.3c}$$

For control constraints $u \in \mathcal{U}$ pointwise, $u(t)$ minimizes $H(\cdot)$ over $\mathcal{U}$ almost everywhere: $u(t) \in \arg\min_{v\in\mathcal{U}} H(y(t),v,p(t),t)$.

Derivation sketch. Consider a perturbation δu and the corresponding linearized state δy solving $\dot{\delta y} = f_y\,\delta y + f_u\,\delta u$, $\delta y(0) = 0$. The first variation is

$$\delta J = \int_0^T (\ell_y\,\delta y + \ell_u\,\delta u)\,\mathrm{d}t + \phi_y(y(T))\,\delta y(T). \tag{3.4}$$

Introduce p and integrate by parts:

$$\delta J = \int_0^T (\ell_y\,\delta y + \ell_u\,\delta u)\,\mathrm{d}t + p(T)^{\mathrm{T}}\delta y(T) - p(0)^{\mathrm{T}}\delta y(0) - \int_0^T \dot{p}^{\mathrm{T}}\delta y\,\mathrm{d}t \tag{3.5}$$

$$- \int_0^T p^{\mathrm{T}}(f_y\,\delta y + f_u\,\delta u)\,\mathrm{d}t. \tag{3.6}$$

Choosing p to satisfy the adjoint ODE in (3.3) with terminal condition cancels all δy terms, leaving

$$\delta J = \int_0^T (\ell_u + f_u^{\mathrm{T}} p)\,\delta u\,\mathrm{d}t = \int_0^T H_u\,\delta u\,\mathrm{d}t, \tag{3.7}$$

so the L^2-gradient of the reduced functional is $g(t) = H_u(y(t),u(t),p(t),t)$.

4 Adjoints for PDE-Constrained Problems

Let y solve a PDE $A(y,u)=0$ in a spatial domain Ω with boundary conditions, and consider $J(y,u)$. The Lagrangian is

$$\mathcal{L}(y,u,p) = J(y,u) + \langle p,\, A(y,u)\rangle_{Y^*,Y}\,, \tag{4.1}$$

with p the PDE adjoint. Taking variations and integrating by parts shifts derivatives from δy to p, producing the *adjoint PDE*. For example, for a linear elliptic control problem

$$-\nabla\cdot(a\nabla y) = Bu + s \qquad \text{in } \Omega, \tag{4.2a}$$

$$y = 0 \qquad \text{on } \partial\Omega, \tag{4.2b}$$

$$J(y,u) = \tfrac{1}{2}\,\|y-y_d\|^2_{L^2(\Omega)} + \tfrac{\alpha}{2}\,\|u\|^2_{L^2(\Omega)}\,, \tag{4.2c}$$

we obtain the adjoint

$$-\nabla\cdot(a\nabla p) = y - y_d \qquad \text{in } \Omega, \tag{4.3a}$$

$$p = 0 \qquad \text{on } \partial\Omega, \tag{4.3b}$$

with reduced gradient

$$\nabla\hat{J}(u) = \alpha u + B^* p. \tag{4.4}$$

If u is box-constrained, the optimality condition becomes a projection: $u = \Pi_{[u_{\min},u_{\max}]}(-\alpha^{-1}B^*p)$.

5 Discrete vs. Continuous Adjoint

Two routes exist:

- Optimize-then-discretize: derive adjoint equations in the continuum and then discretize them.
- Discretize-then-optimize: discretize the state equation and objective, then derive the adjoint of the resulting discrete system.

They agree when discretization commutes with differentiation (e.g., Galerkin consistency) and objectives are discretized compatibly. For time-dependent problems, the *discrete adjoint* corresponds to reverse-time propagation through the time-stepping scheme and yields gradients consistent with the discrete objective.

6 Adjoint and Algorithmic Differentiation

Reverse-mode algorithmic differentiation (AD) is a discrete adjoint of a computational graph. Backpropagation in deep learning is the adjoint of forward propagation. Many PDE codes can be differentiated via AD frameworks or specialized libraries (e.g., `dolfin-adjoint/fenics-adjoint` for FEniCS), automatically generating adjoint solves and gradients.

7 Inequality Constraints and KKT

For control constraints $u \in \mathcal{U}$ (e.g., box constraints), the stationarity becomes a variational inequality

$$\left\langle \nabla \hat{J}(u), v - u, \right\rangle \geq 0 \quad \forall v \in \mathcal{U}, \tag{7.1}$$

which in Hilbert spaces leads to the projection formula $u = \Pi_{\mathcal{U}}\big(u - \tau \nabla \hat{J}(u)\big)$ for $\tau > 0$. With state inequality constraints (e.g., $g(y) \leq 0$), additional multipliers and complementarity conditions arise, and the adjoint gains additional terms $g_y^* \mu$ in its right-hand side.

8 Second-Order Information and Newton-Krylov

The reduced Hessian action $\nabla^2 \hat{J}(u)[\delta u]$ can be computed via an *incremental state* and an *incremental adjoint*. Linearize state and adjoint around (y, u, p), solve the incremental systems for $(\delta y, \delta p)$, and assemble the Hessian-vector product. This enables matrix-free Newton-Krylov methods with adjoint-based preconditioners.

9 Algorithmic Pattern

A typical gradient-based loop with adjoints:

1. Given u, solve the state equation for y.

2. Assemble and solve the adjoint equation for p using (y, u).

3. Form the reduced gradient $g(u)$ (e.g., $g = J_u + C_u^* p$).

4. Update control: projected gradient, L-BFGS, or Newton step with line search.

5. Stop on stationarity and feasibility criteria.

For transient problems, use checkpointing to manage memory during reverse-time adjoint integration.

10 Worked Examples

10.1 Linear-Quadratic Regulator (discrete time)

Consider $y_{k+1} = Ay_k + Bu_k$, y_0 given, and cost

$$J = \sum_{k=0}^{N-1} \left(\tfrac{1}{2} y_k^{\mathrm{T}} Q y_k + \tfrac{1}{2} u_k^{\mathrm{T}} R u_k \right) + \tfrac{1}{2} y_N^{\mathrm{T}} Q_f y_N. \tag{10.1}$$

The discrete adjoint recursion is

$$\lambda_N = Q_f y_N, \tag{10.2a}$$

$$\lambda_k = Q y_k + A^{\mathrm{T}} \lambda_{k+1}, \quad k = N-1, \ldots, 0. \tag{10.2b}$$

The reduced gradient is $g_k = Ru_k + B^{\mathrm{T}}\lambda_{k+1}$. Setting $g_k = 0$ yields the optimality system; with box constraints, use $u_k = \Pi_{[u_{\min},u_{\max}]}\big(-R^{-1}B^{\mathrm{T}}\lambda_{k+1}\big)$.

10.2 Elliptic PDE Control (continuous time)

The example in Eqs. (4.2) and (4.3) leads to a forward elliptic solve for y, a backward (same operator) elliptic solve for p, and a cheap gradient assembly. This structure is typical: adjoints often reuse the same left-hand side operator as the state (or its transpose), enabling efficient shared preconditioning.

11 Repository-Specific Mapping (External Mesh Ex3)

We summarize how the adjoint method appears in the provided codebase and how the continuous formulas map to the implemented discrete operators.

11.1 Model and Spaces

- **Domain**: Ω from an external XDMF mesh; control region $\omega \subset \Omega$ is given by a cell tag with integer id ω_{id}.
- **State/adjoint space** V: Lagrange $\mathbb{P}_1$ with homogeneous Dirichlet boundary conditions.
- **Control space** U: realized on the same $\mathbb{P}_1$ basis but endowed with the L^2 inner product via a mass matrix M_U; no boundary conditions are applied in U.

11.2 Discrete Operators

Let $\{\varphi_i\}_{i=1}^n$ be the nodal $\mathbb{P}_1$ basis on V. The code assembles

$$A_{ij} = \int_\Omega \nabla\varphi_i \cdot \nabla\varphi_j \,\mathrm{d}x, \qquad \text{(stiffness)}, \tag{11.1}$$

$$M_{ij} = \int_\Omega \varphi_i\,\varphi_j \,\mathrm{d}x, \qquad \text{(state mass)}, \tag{11.2}$$

$$(M_U)_{ij} = \int_\Omega \varphi_i\,\varphi_j \,\mathrm{d}x, \qquad \text{(control mass)}, \tag{11.3}$$

$$B_{ij} = \int_\omega \varphi_i\,\varphi_j \,\mathrm{d}x. \qquad \text{(coupling on } \omega\text{)} \tag{11.4}$$

Dirichlet boundary conditions are applied to the operator A (rows/columns modified in the assembled FEM operator), and right-hand sides used with A have boundary DOFs zeroed. No boundary treatment is applied to B or M_U.

11.3 State, Adjoint, and Objective

The discrete state $y \in \mathbb{R}^n$ for a control $u \in \mathbb{R}^n$ solves

$$A\,y = B\,u \quad \text{(homogeneous Dirichlet via RHS zeroing at boundary DOFs).} \tag{11.5}$$

The target is a pyramid-like profile $y_d(x) = 0.5 - \max\{|x - 0.5|, |y - 0.5|\}$ interpolated onto V. The discrete objective is

$$F(u) = \tfrac{1}{2}(y - y_d)^{\mathrm{T}} M (y - y_d) + \tfrac{\beta}{2}\, u^{\mathrm{T}} M_U\, u. \tag{11.6}$$

The adjoint $p \in \mathbb{R}^n$ solves

$$A\, p = M\,(y - y_d), \quad \text{with the same Dirichlet treatment as the state.} \tag{11.7}$$

11.4 Reduced Gradient and Hessian

With the inner product induced by M_U on the control space, the gradient routine computes

$$\nabla F(u) = M_U^{-1} B^{\mathrm{T}} p + \beta\, u. \tag{11.8}$$

This is the Riesz-represented gradient in U such that $\langle \nabla F(u),\, v\rangle_U = \langle F'(u),\, v\rangle$ for all v. The (matrix-free) Hessian action implemented in the code is

$$H\, v = M_U^{-1} B^{\mathrm{T}} A^{-1} M\, A^{-1} B\, v + \beta\, v, \tag{11.9}$$

which is consistent with linearizing the state and adjoint equations around (y, u, p).

11.5 Algorithmic Details and Verification

- **Stepsizes:** Gradient descent uses $\alpha = 1/L$ with L estimated by a power iteration on H; Barzilai–Borwein alternates BB1/BB2 formulas using the M_U-induced inner products.
- **Boundary handling:** Apply Dirichlet boundary conditions only to A; do not modify M_U or B. Zero boundary DOFs on right-hand sides passed to A.
- **Consistency check:** Verify gradients via directional derivatives in the U inner product: compare $\frac{F(u+\epsilon v) - F(u)}{\epsilon}$ and $\langle \nabla F(u),\, v\rangle_U$ for random v.

Interpretation. In the continuous setting for $-\Delta y = \chi_\omega u$ with Dirichlet BCs and objective $\frac{1}{2}\,\|y - y_d\|^2_{L^2(\Omega)} + \frac{\beta}{2}\,\|u\|^2_{L^2(\Omega)}$, the adjoint solves $-\Delta p = y - y_d$ with homogeneous Dirichlet boundary conditions, and the reduced gradient is $g = \beta u + B^* p$ where B^* is the L^2 adjoint of B (multiplication by χ_ω followed by the appropriate Riesz map). The discrete formulas above implement these relationships faithfully with FEM matrices.

12 Boundary/Terminal Conditions and Transversality

For ODEs, terminal costs $\phi(y(T))$ generate terminal conditions $p(T) = \phi_y(y(T))$. For PDEs, integration by parts introduces boundary terms; natural boundary conditions for the adjoint depend on the objective and state BCs. For instance, Dirichlet state BCs typically induce homogeneous Dirichlet adjoint BCs when the objective measures interior misfit; Neumann-type objectives can induce nonzero adjoint Neumann data.

13 Practical Tips

- **Consistency:** Match discretization of the objective with the discretization of the forward model to ensure consistent gradients.
- **Scaling:** Nondimensionalize and scale J so state and control terms balance; improves conditioning.
- **Linear solvers:** Reuse factorizations or preconditioners between state and adjoint solves when operators coincide.
- **Checkpointing:** For long transients, use recomputation/checkpointing to trade time for memory during reverse sweeps.
- **Verification:** Check gradients via directional derivatives: $\frac{\hat{J}(u+\epsilon v)-\hat{J}(u)}{\epsilon}$ vs. $\left\langle \nabla \hat{J}(u),\, v \right\rangle$.

14 Summary

The adjoint method is the workhorse for efficient gradient computation in differential equation–constrained optimization. By introducing a Lagrange multiplier and transferring derivatives, adjoints deliver gradients at a cost comparable to one additional solve, enabling scalable optimization, calibration, and control in high-dimensional systems.

References

References

[1] M. Hinze, R. Pinnau, M. Ulbrich, S. Ulbrich, *Optimization with PDE Constraints*, Springer, 2009.

[2] J. L. Troutman, *Variational Calculus and Optimal Control*, Springer, 1996.

[3] M. B. Giles, N. A. Pierce, "An introduction to the adjoint approach to design," *Flow, Turbulence and Combustion*, 65, 2000.

[4] A. Borzì, V. Schulz, *Computational Optimization of Systems Governed by PDEs*, SIAM, 2012.